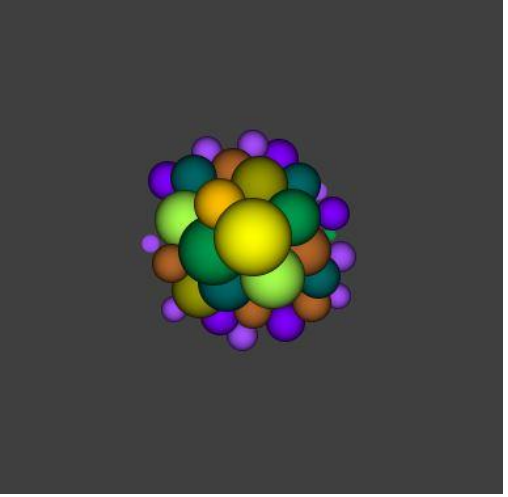
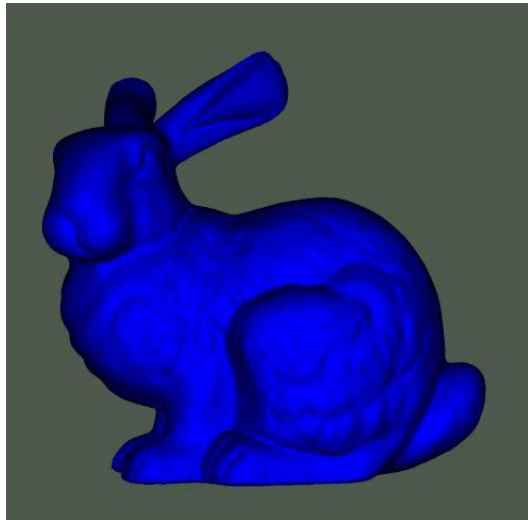


CS 440: Computer Graphics

Week 7

Lecture 13: Mesh Analysis



Acknowledgements: Cem Yuksel, University of Utah; Waqar Saleem, Habib University

Announcements and Recap

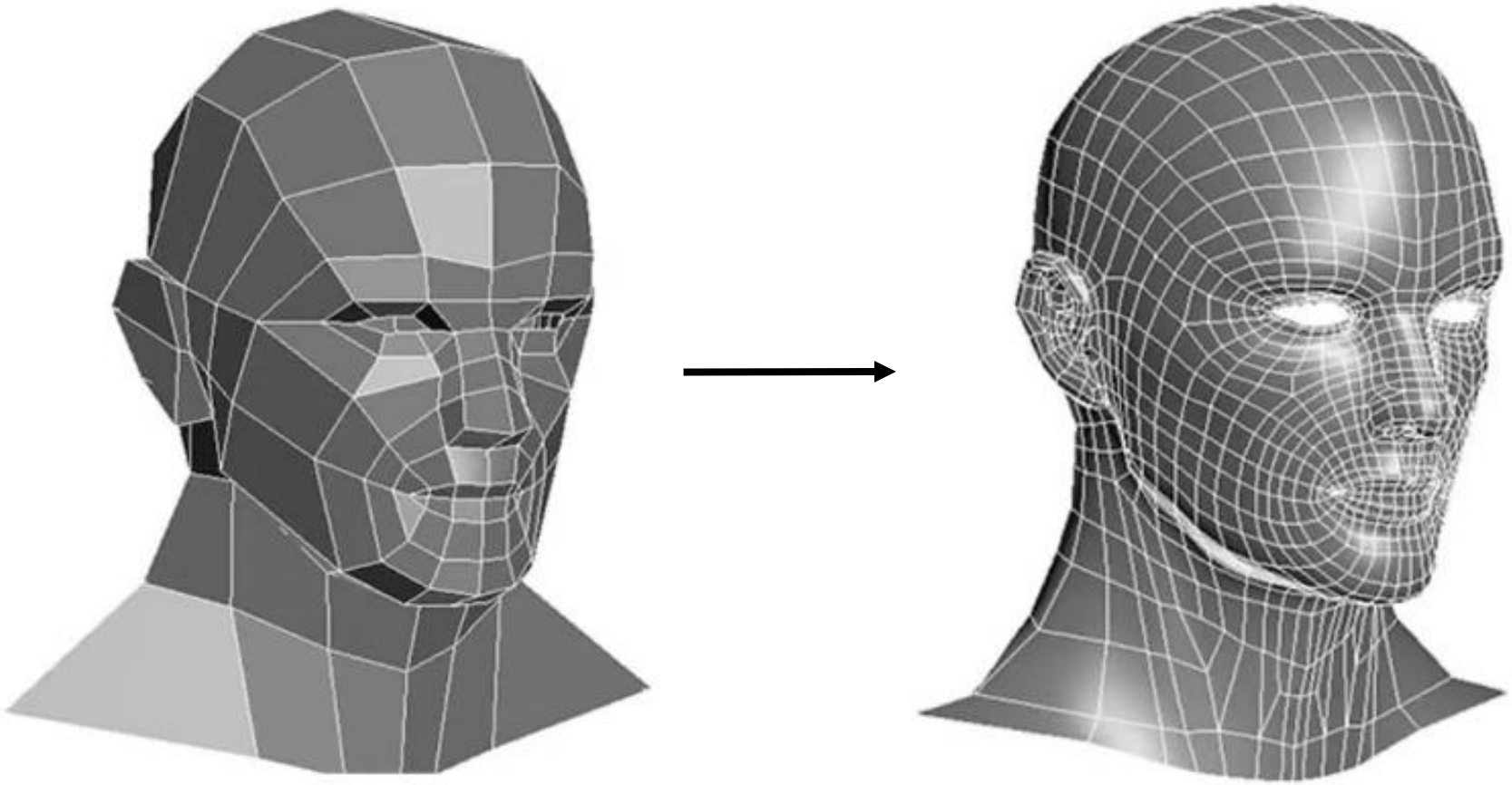
- Quiz today
- Midterm next week on Wednesday
- Assignment 3 due Friday

- Recap:
  - Mesh Data Structures
  - Subdivision

Today: Mesh Analysis

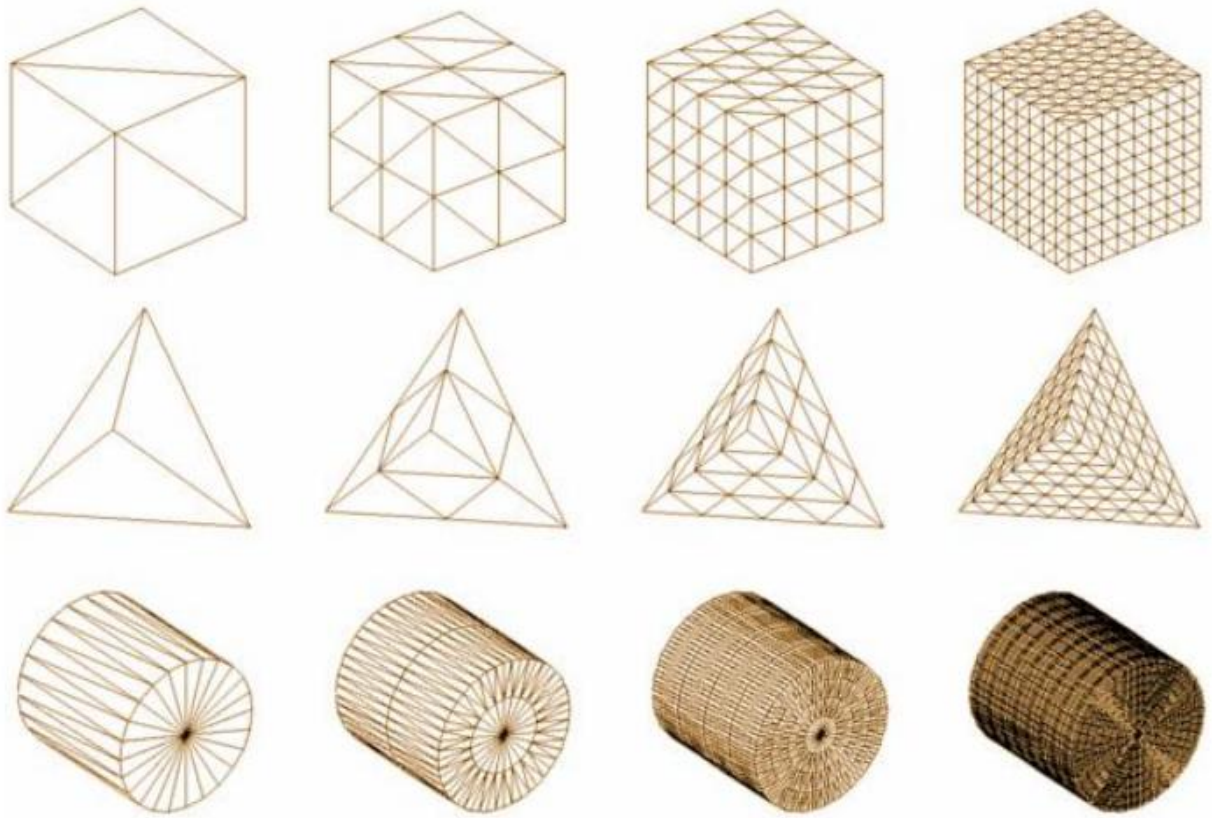
- Mesh Analysis
  - Subdivision
  - Smoothing
- Platonic Solids
- Viewing
  - Viewing Projections
  - General Viewing

Subdivision



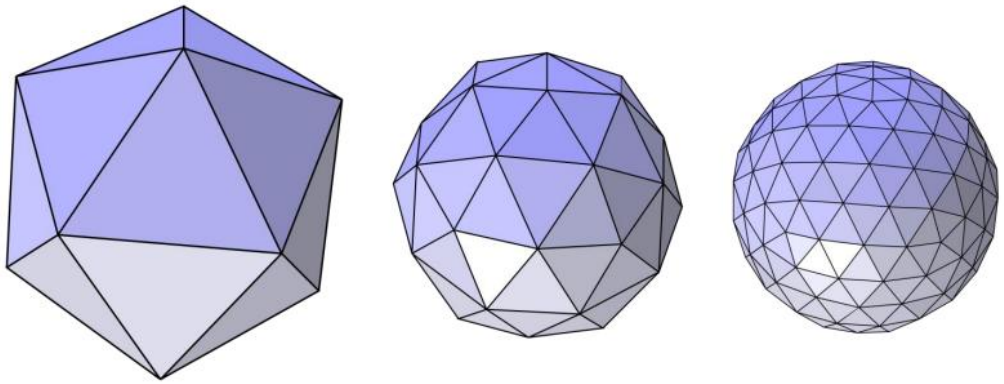
Linear Subdivision

- Insert vertices at edge midpoints
- Adds faces, but doesn't change shape



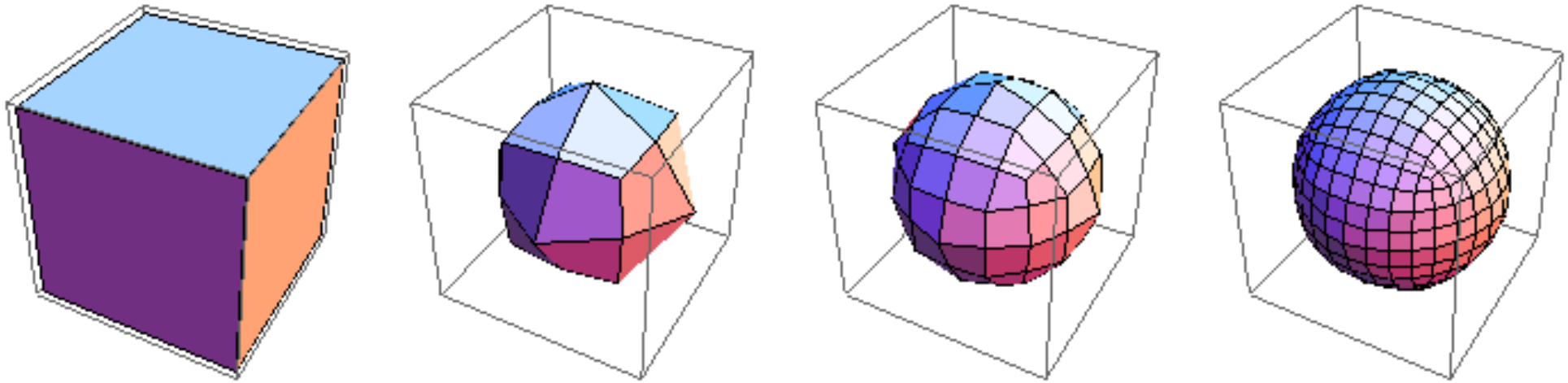
Loop Subdivision

- Works for triangular meshes
- Algorithm:
  - Split each triangle into four triangles
  - Move the new vertices (using the weighted average of vertex neighbors)
  - Move old vertices based on the weighted average of its position and its neighbor's positions (*vertex valence*)
  - Make smaller triangles



Catmull-Clark Subdivision

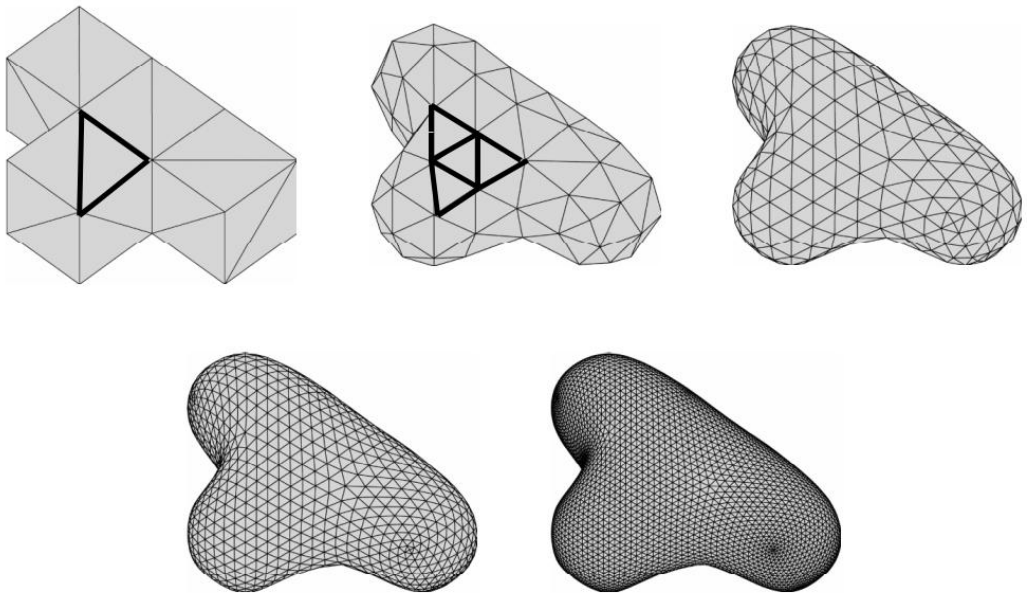
- Mostly works on quad meshes
- How it works: https://youtu.be/2UjfxAuXuSU?si=qePilYVBlBb1VaMD

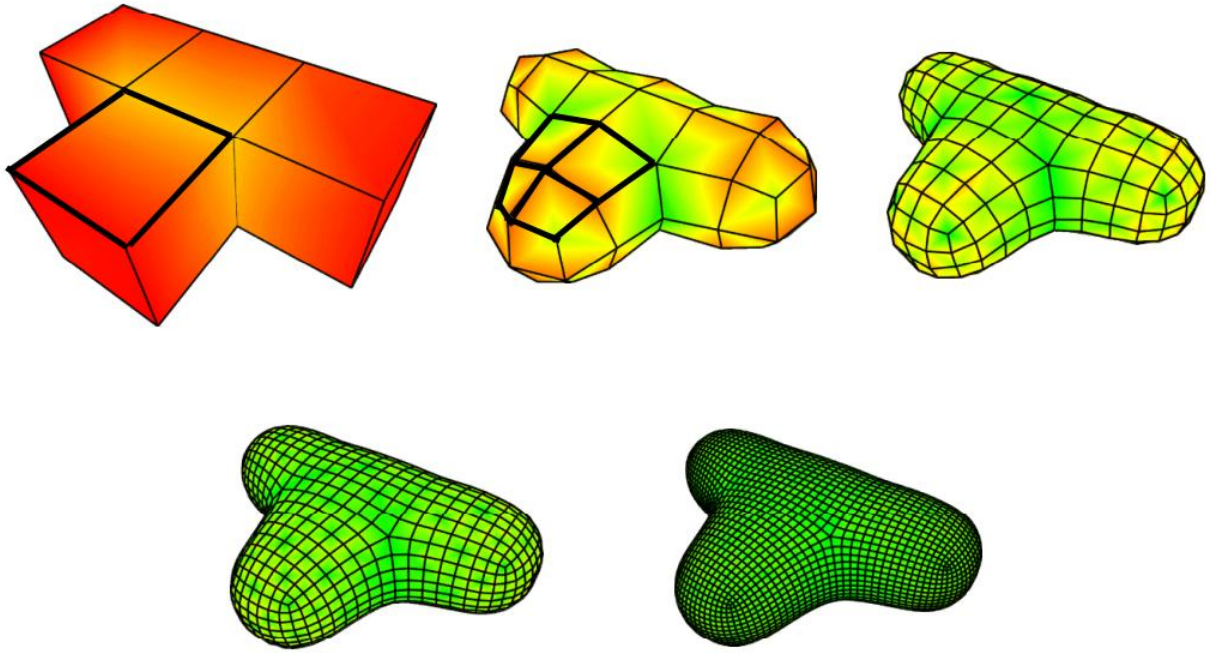


Catmull-Clark Subdivision: The Algorithm

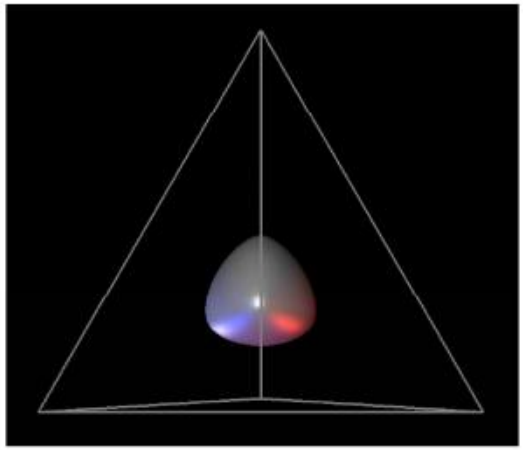
- For each face, add a new vertex called *face point*
  - New vertex is the average of all original vertices of that face
- For each edge, add a new vertex called *edge point*
  - Average of the two neighboring face points and two vertices of edge
- Move old vertices to new positions
  - Use weighted average, based on *valence* of a vertex
- Connect the new points
  - New face point with new edge point
  - New vertices (generated in previous step) with new edge points

Loop vs Catmull-Clark Subdivision

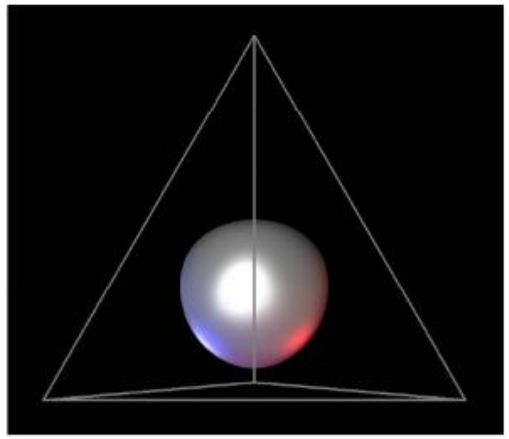




Loop vs Catmull Clark Subdivision



Loop

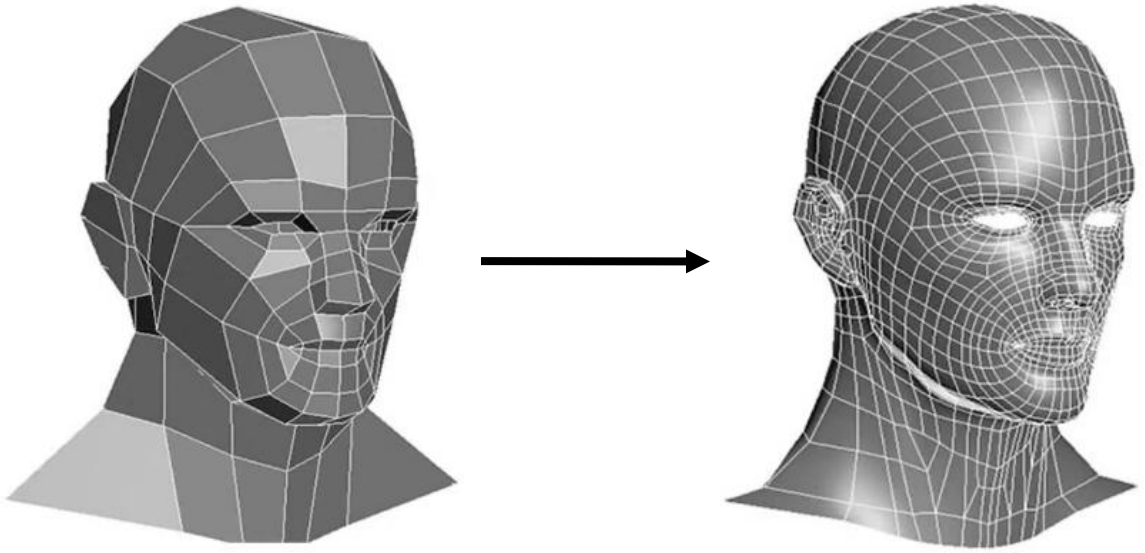


Catmull-Clark

Initial mesh *Loop* *Catmull-Clark* *Catmull-Clark,after triangulation*

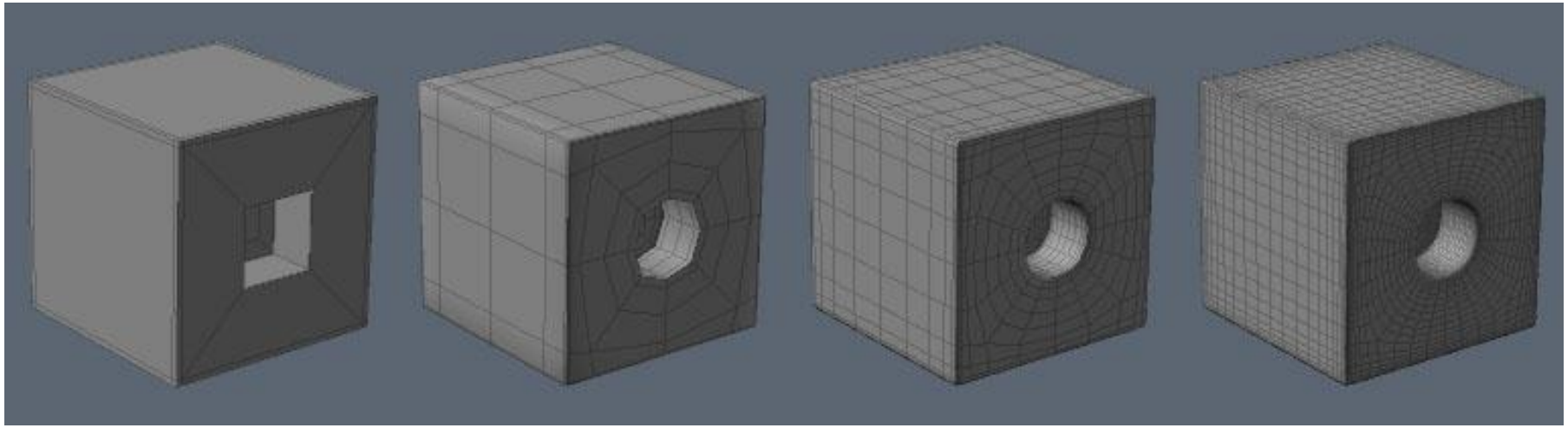
3D Modeling and Subdivision

- To model high resolution meshes, we typically start with low resolution first
- Then we apply subdivision schemes to it



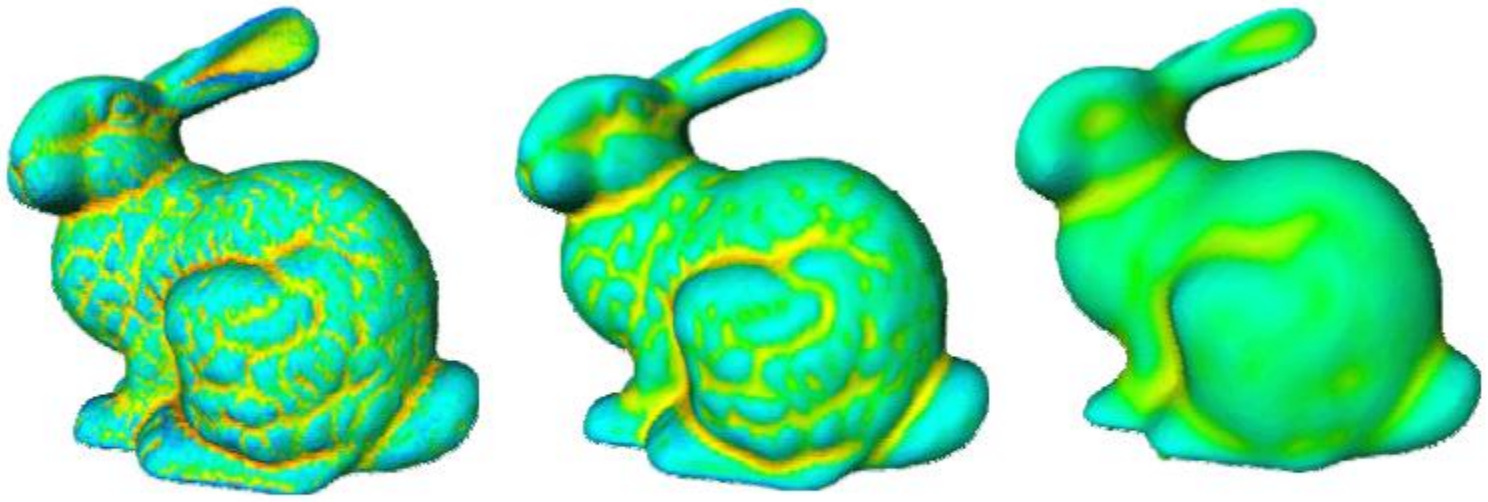
Subdivision Surfaces: Challenges

- Dealing with creases and sharp corners
  - Do we really need to smooth all edges?
- Dealing with boundaries
- Dealing with *extraordinary* vertices



Mesh Smoothing

- Reduces “noise” from meshes, improves visual quality
- Moves vertices according to its local neighborhood



0 Iterations 5 Iterations 20 Iterations

Laplacian Smoothing

- Inspired by finite difference approximation of Laplacian
- Algorithm:
  - Calculate the centroid of neighborhood vertices
  - Apply the Laplacian operation: $\Delta v = v_c - v$
  - Smoothing step: Move vertices to their new position $v^{new} = v^{old} - \lambda\, \Delta v$
- λ is the smoothing factor, where $0 \leq \lambda \leq 1$

- What happens if we do this repeatedly?